VECTOR

MICROSAR SOME/IP Transport Protocol

Technical Reference

Version 1.0.1

Authors	Supraja Kovuru, Anthony Thomas
Status	Released

Document Information

History

Author	Date	Version	Remarks
Supraja Kovuru	2018-01-19	1.0.0	Initial Version
Anthony Thomas	2018-02-13	1.0.1	Added section showing how to to transmit and receive data without the RTE API.

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_SOMEIPTransportProtocol.pdf	4.3.0
[2]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_TR_SomeIpExample.pdf	4.2.1
[4]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	4.3.0
[5]	Vector	TechnicalReference_SocketAdaptor.pdf	8.4.0 or later

Scope of the Document

This technical reference describes the general use of the SomeIpTp basis software.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 6

2 Introduction 7
2.1 Architecture Overview 8

3 Functional Description 10
3.1 Features 10
3.1.1 Deviations 10
3.1.2 Additions/ Extensions 10
3.1.2.1 Transmit Queue 11
3.1.2.2 Monitoring Transmission Confirmation timeout 11
3.2 Initialization 11
3.3 States 11
3.4 Main Functions 11
3.5 Error Handling 12
3.5.1 Development Error Reporting 12
3.5.2 Runtime Error Reporting 13

4 Integration 14
4.1 Scope of Delivery 14
4.1.1 Static Files 14
4.1.2 Dynamic Files 14
4.1.3 User modifiable templates 14
4.2 Include Structure 15
4.3 Critical Sections 15

5 API Description 17
5.1 Services provided by SomeIpTp 17
5.1.1 SomeIpTp_GetVersionInfo 17
5.1.2 SomeIpTp_InitMemory 17
5.1.3 SomeIpTp_Init 18
5.1.4 SomeIpTp_Transmit 18
5.1.5 SomeIpTp_MainFunctionTx 19
5.1.6 SomeIpTp_MainFunctionRx 19
5.2 Services used by SomeIpTp 20
5.3 Callback Functions 20
5.3.1 SomeIpTp_TriggerTransmit 20
5.3.2 SomeIpTp_TxConfirmation 21
5.3.3 SomeIpTp_RxIndication 22

5.4 Configurable Interfaces 22

6 Configuration 23
6.1 Configuration Variants 23
6.2 Additional configuration hints 23
6.2.1 SomeIpTpRxTimeoutTime 23
6.2.2 SomeIpTpTxConfirmationTimeoutTime 23
6.2.3 Sending and transmitting data without the RTE API 23
6.2.3.1 Configuring the LdCom with the DaVinci Configurator 23
6.2.3.2 Adjusting the templates 25
6.2.3.3 Reading and Writing data from the SWC 25

7 Glossary and Abbreviations 27
7.1 Glossary 27
7.2 Abbreviations 27

8 Contact 28

Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview 8
Figure 2-2 Interfaces to adjacent modules of the SomeIpTp 9
Figure 4-4-1 Include Structure 15
Figure 6-1 Configuration of the LdCom's I-PDU used during reception. 24
Figure 6-2 Configuration of the LdCom's I-PDU used during transmission. 25

Tables

Table 1-1 Component history 6
Table 3-1 Supported AUTOSAR standard conform features 10
Table 3-2 Not supported AUTOSAR standard conform features 10
Table 3-3 Features provided beyond the AUTOSAR standard 11
Table 3-4 Service IDs 12
Table 3-5 Development errors reported to DET 12
Table 3-6 Runtime errors reported to DET 13
Table 4-1 Static files 14
Table 4-2 Generated files 14
Table 4-3 User modifiable templates 14
Table 5-1 SomeIpTp_GetVersionInfo 17
Table 5-2 SomeIpTp_InitMemory 18
Table 5-3 SomeIpTp_Init 18
Table 5-4 SomeIpTp_Transmit 19
Table 5-5 SomeIpTp_MainFunctionTx 19
Table 5-6 SomeIpTp_MainFunctionRx 20
Table 5-7 Services used by the SomeIpTp 20
Table 5-8 SomeIpTp_TriggerTransmit 21
Table 5-9 SomeIpTp_TxConfirmation 21
Table 5-10 SomeIpTp_RxIndication 22
Table 6-1 Functions intended to be used by the SWC. 25
Table 7-1 Glossary 27
Table 7-2 Abbreviations 27

© 2018 Vector Informatik GmbH

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00	First beta version that provides the possibility to segment SOME/IP packets

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module SomeIpTp as specified in [1].

Supported AUTOSAR Release*:	4.3.0	
Supported Configuration Variants:	pre-compile, post-build	
Vendor ID:	SomeIpTp_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	SomeIpTp_MODULE_ID	177 decimal (according to ref. [4])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The SomeIpTp module based on AUTOSAR SOME/IP Transport Protocol provides the following services :

- ➔ Segments SOME/IP packets which do not fit into one single UDP packet.
- ➔ Reassembles the received SOME/IP segments.
- ➔ Detection of errors during segmentation and reassembly.

2.1 Architecture Overview

The following figure shows where the SomeIpTp is located in the AUTOSAR architecture.

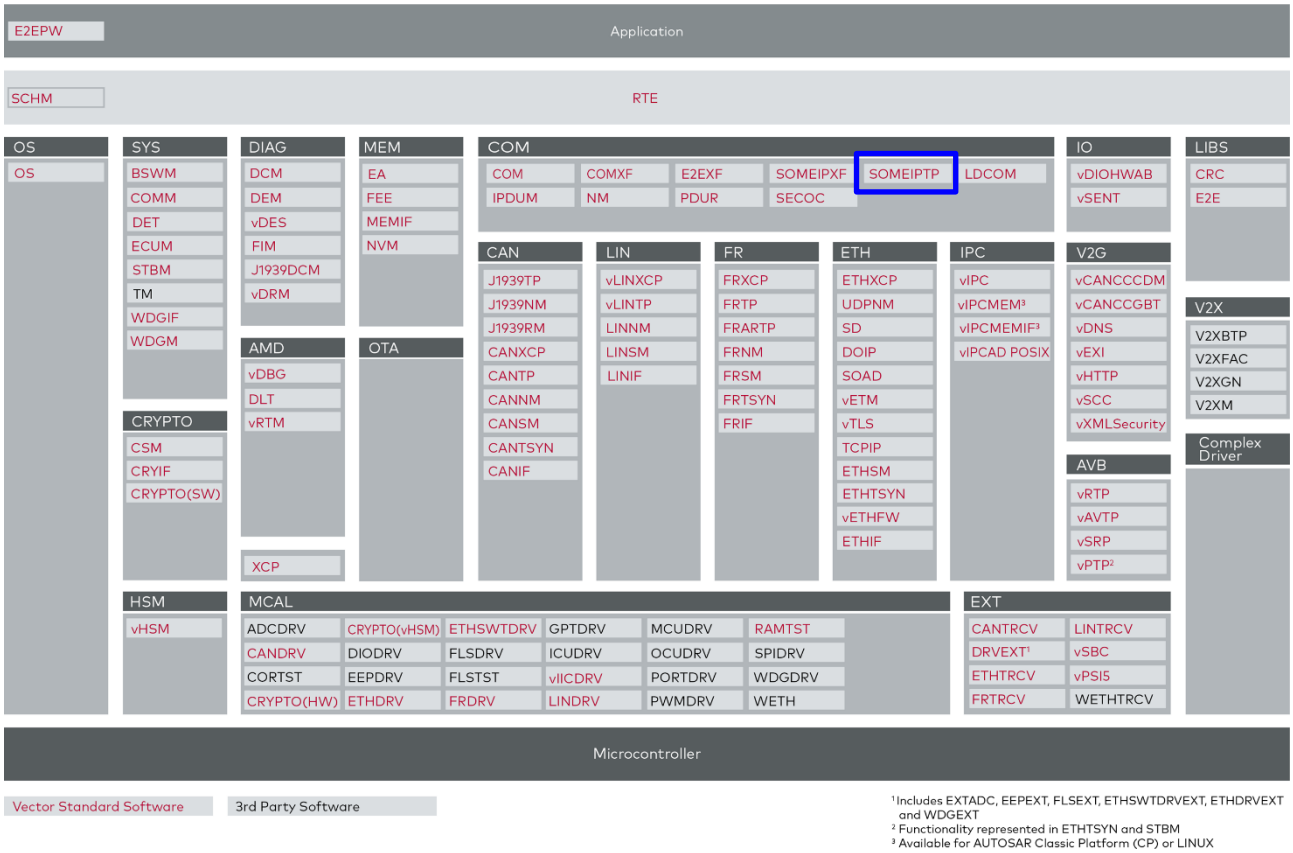


Figure 2-1 AUTOSAR 4.3 Architecture Overview

The next figure shows the interfaces to adjacent modules of the SomeIpTp. These interfaces are described in chapter 5.

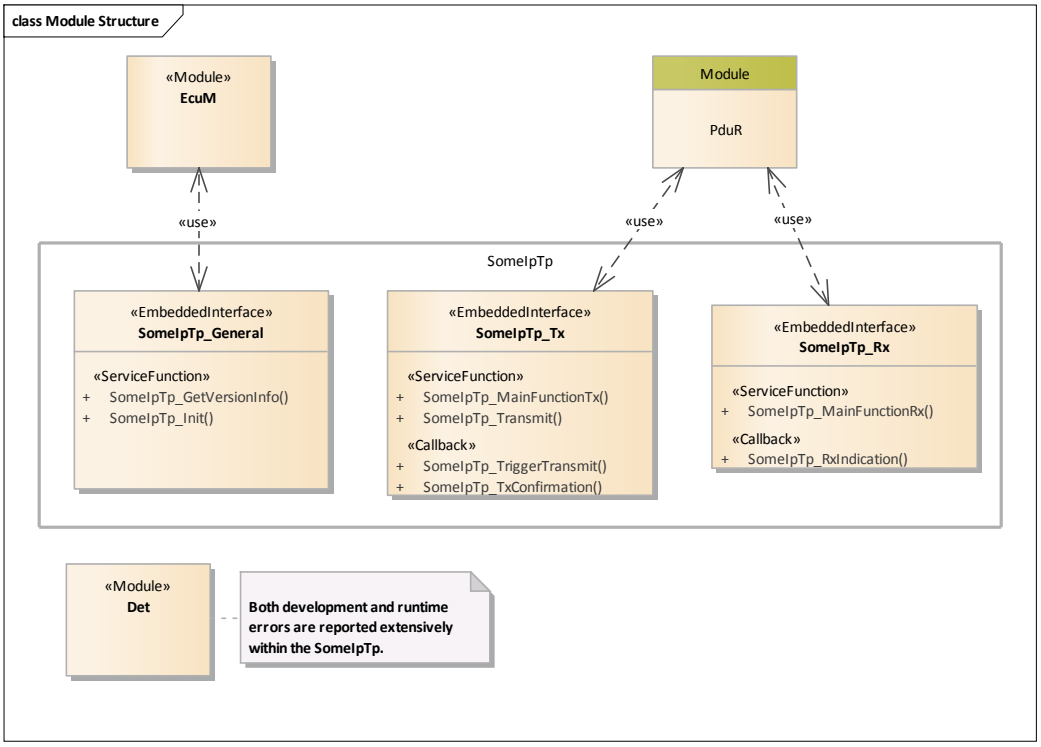


Figure 2-2 Interfaces to adjacent modules of the SomeIpTp

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the SomeIpTp.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further SomeIpTp functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-3 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Segmented and unsegmented data transmission
Segmented and unsegmented reception
Supervision of timeouts
Detection of errors during segmentation and reassembly

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features	ASR Version
Forwarding of Metadata during transmission and reception.	4.3.0
Negative TX Confirmation is not supported. For more details refer Monitoring Transmission Confirmation timeout	4.3.0

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Transmit queue

Features Provided Beyond The AUTOSAR Standard
Monitoring Transmission Confirmation Timeout

Table 3-3 Features provided beyond the AUTOSAR standard

3.1.2.1 Transmit Queue

If several TX SDUs are requested by the upper layer, they will be added to a queue. Supervision of timeouts and transmission of further segments of these SDUs is based on the order of elements stored in the Queue.

3.1.2.2 Monitoring Transmission Confirmation timeout

To be compatible with the other modules, `SomeIpTp_TxConfirmation()` is implemented only with the parameter `TxPduId`. Since negative TX Confirmation is currently not supported, if there is no call to `SomeIpTp_TxConfirmation()` within a specified time, the segmentation process for the corresponding TX SDU is aborted.

In the Vector SomeIpTp, this timeout is configurable by the parameter `SomeIpTpTxConfirmationTimeoutTime` per `SomeIpTpChannel`.

3.2 Initialization

The API `SomeIpTp_Init()` uses the given configuration set to initialize all global variables of SomeIpTp and brings each SDU to state Idle, where segmentation and reassembly can be started.

3.3 States

After initialization, all the TX and RX SDUs are set to Idle (`SOMEIPTP_TX_STATUS_IDLE`, `SOMEIPTP_RX_STATUS_IDLE`). Segmentation or reassembly is possible only in this state.

During the segmentation process, the TX SDU state is either wait for confirmation (`SOMEIPTP_TX_STATUS_WAIT_TXCONF_SF`) or wait for separation time (`SOMEIPTP_TX_STATUS_WAIT_ST`). The state is reset to Idle if the disassembly process is aborted or is finished.

During the reassembly process, the RX SDU state is set to wait for next segment (`SOMEIPTP_RX_STATUS_WAIT_NEXT_SF`). The state is reset to Idle after the complete frame is received or reassembly process is interrupted.

3.4 Main Functions

The functions `SomeIpTp_MainfunctionTx()` and `SomeIpTp_MainfunctionRx()` perform all the tasks that should be done cyclically.

The timing parameters like `SomeIpTpSeparationTime` and `SomeIpTpRxTimeoutTime` are calculated based on the configured `SomeIpTpTxMainfunctionPeriod` and `SomeIpTpRxMainfunctionPeriod` respectively.

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `SOMEIPTP_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported SomeIpTp ID is 177.

The reported service IDs identify the services which are described in 5.1. The following table presents the service IDs and the related services:

Service ID	Service
0x01	SomeIpTp_GetVersionInfo
0x02	SomeIpTp_Init
0x03	SomeIpTp_MainfunctionTx
0x04	SomeIpTp_MainfunctionRx
0x40	SomeIpTp_TxConfirmation
0x41	SomeIpTp_TriggerTransmit
0x42	SomeIpTp_RxIndication
0x49	SomeIpTp_Transmit

Table 3-4 Service IDs

The errors reported to DET are described in the following table:

Error Code		Description
0x01	SOMEIPTP_E_NOTINIT	SomeIpTp not Initialized
0x02	SOMEIPTP_E_PARAM_POINTER	API called with wrong PduInfo pointer
0x03	SOMEIPTP_E_PARAM	API called with wrong parameters
0xFF	SOMEIPTP_E_FATAL_ERROR	Transmit Queue is full

Table 3-5 Development errors reported to DET

3.5.2 Runtime Error Reporting

By default, Runtime related errors are reported to the to the DET using the service `Det_ReportRuntimeError()` as specified in [2], if runtime error reporting is enabled (i.e. pre-compile parameter `SOMEIPTP_RUNTIME_ERROR_REPORT==STD_ON`).

If another module is used for runtime error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportRuntimeError()`.

The runtime errors reported to DET are described in the following table:

Error Code		Description
0x04	SOMEIPTP_E_MESSAGE_TYPE	TP Flag not set
0x05	SOMEIPTP_E_INCONSISTENT_SEQUENCE	Out of order segment received
0x06	SOMEIPTP_E_INCONSISTENT_HEADER	Different header compared to previous segments
0x07	SOMEIPTP_E_DISASSEMBLY_INTERRUPT	Segment received with incorrect alignment or insufficient buffer reported by upper layer
0x08	SOMEIPTP_E_ASSEMBLY_INTERRUPT	Requested payload not available by the upper layer

Table 3-6 Runtime errors reported to DET

4 Integration

This chapter gives necessary information for the integration of the MICROSAR SomeIpTp into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the SomeIpTp contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
SomeIpTp.c	Source file of SomeIpTp
SomeIpTp.h	Header file of SomeIpTp
SomeIpTp_Cbk.h	Header file with SomeIpTp callback function prototypes
SomeIpTp_Priv.h	Header file with internal SomeIpTp definitions.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool [config tool].

File Name	Description
SomeIpTp_Cfg.h	Header file for configuration parameters (e.g. Pre-compile switches)
SomeIpTp_LCfg.c	Source file for generated source code
SomeIpTp_LCfg.h	Header file for generated source code
SomeIpTp_PBCfg.c	Header file for generated source code (post-build-time configurable parameters)
SomeIpTp_PBCfg.h	Header file for generated source code (post-build-time configurable parameters)
SomeIpTp_Types.h	Header file containing SomeIpTp type definitions

Table 4-2 Generated files

4.1.3 User modifiable templates

File Name	Description
Cdd_SomeIpTp.c	Example implementation of a fictive component located between the LdCom and the SWC, that allows the transmission and reception of byte streams bypassing the RTE. This file can be adapted as required.
Cdd_SomeIpTp.h	Example header of the fictive component. This file can be adapted as required.

Table 4-3 User modifiable templates

4.2 Include Structure

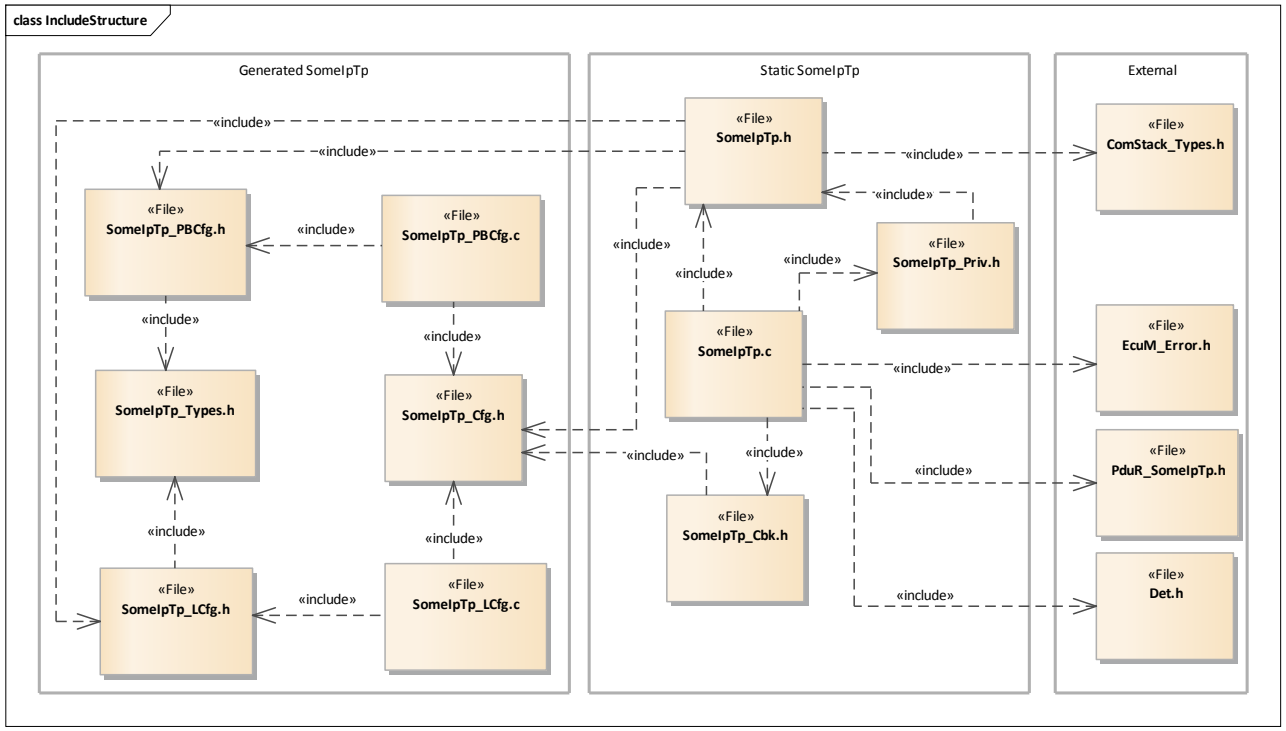


Figure 4-4-1 Include Structure

Note
The only files that need to be included by other modules and software components are `SomeIpTp.h` and `SomeIpTp_Cbk.h`. These files contain all definitions and inclusions required to use the SomeIpTp.

4.3 Critical Sections

All services and callbacks for transmission, reception and state changes of SomeIpTp may be called in interrupt or task level. This, a synchronization mechanism is implemented to guarantee data consistency.

The synchronization mechanism is defined by AUTOSAR covers the entering and leaving of so called critical sections.

The implementation of the critical sections must avoid that multiple relevant tasks or interrupt service routines can enter each of the critical sections more than once at the same time.

Relevant interrupt services in the SomeIpTp context are interrupt services originated from physical bus events (Ethernet, CAN, FlexRay, LIN etc.).

Relevant tasks in the SomeIpTp context are all tasks which call SomeIpTp APIs. Usually these tasks are limited to the tasks on which other BSW modules (SoAd, LdCom etc.).

© 2018 Vector Informatik GmbH

The SomeIpTp defined the following exclusive area.

> SOMEIPTP_EXCLUSIVE_AREA_0 is used to protect Transmit Queue access and memory access to TX SDU information from interrupting calls to services and callbacks of SomeIpTp.

For implementing this critical section it could be sufficient to

> Disable all bus relevant interrupts related to calls of SomeIpTp APIs.

> Disable all Ethernet bus relevant interrupts if all the modules calling SomeIpTp APIs are mapped to one task.

5 API Description

For an interfaces overview please see Figure 2-2.

5.1 Services provided by SomeIpTp

5.1.1 SomeIpTp_GetVersionInfo

Prototype	
`void` SomeIpTp_GetVersionInfo`( StdVersionInfoType *VersionInfo )`	
Parameter	
`parameter`	Pointer to store the version information.Parameter must not be NULL.
Return code	
`void`	none
Functional Description	
Returns the version information.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant.	
Expected Caller Context	
> TASK|ISR2	

Table 5-1 SomeIpTp_GetVersionInfo

5.1.2 SomeIpTp_InitMemory

Prototype	
`void` SomeIpTp_InitMemory`( void )`	
Parameter	
`void`	none
Return code	
`void`	none
Functional Description	
Initializes *_INIT_* variables. This is used in case they are not initialized by startup code.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is Non-reentrant.	

Expected Caller Context
> TASK

Table 5-2 SomeIpTp_InitMemory

5.1.3 SomeIpTp_Init

Prototype	
`void` SomeIpTp_Init`( const SomeIpTp_ConfigType *SomeIpTp_ConfigPtr )`	
Parameter	
`SomeIpTp_ConfigPtr [in]`	Configuration structure for initializing the module.
Return code	
`void`	none
Functional Description	
This function initializes the module SomeIpTp. It initializes all the variables and sets the module to initialized.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > Interrupts are disabled and SomeIpTp_InitMemory has been called unless SomeIpTp_ModuleInitialized is initialized by startup code. > This function is synchronous. > This function is non-reentrant.	
Expected Caller Context	
> TASK	

Table 5-3 SomeIpTp_Init

5.1.4 SomeIpTp_Transmit

Prototype	
`Std_ReturnType` SomeIpTp_Transmit`( PduIdType TxPduId, const PduInfoType *PduInfoPtr )`	
Parameter	
`TxPduId [in]`	Configuration structure for initializing the module.
`PduInfoPtr[in]`	Length of and pointer to the PDU data.
Return code	
`Std_ReturnType`	E_OK Transmit request was accepted.
	E_NOT_OK Transmit request was not accepted.

Functional Description
Requests transmission of a PDU.
Particularities and Limitations
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant.
Expected Caller Context
> TASK|ISR2

Table 5-4 SomeIpTp_Transmit

5.1.5 SomeIpTp_MainFunctionTx

Prototype	
`void` SomeIpTp_MainFunctionTx `( void )`	
Parameter	
`void`	none.
Return code	
`void`	none.
Functional Description	
Schedules transmission of consecutive segments and monitors TX Confirmation timeout	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.	
Expected Caller Context	
> TASK	

Table 5-5 SomeIpTp_MainFunctionTx

5.1.6 SomeIpTp_MainFunctionRx

Prototype	
`void` SomeIpTp_MainFunctionRx `( void )`	
Parameter	
`void`	none.
Return code	
`void`	none.

Functional Description
This function monitors RxTimeoutTime
Particularities and Limitations
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant.
Expected Caller Context
> TASK

Table 5-6 SomeIpTp_MainFunctionRx

5.2 Services used by SomeIpTp

In the following table services provided by other components, which are used by the SomeIpTp are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
DET	Det_ReportRuntimeError
PDUR	PduR_SomeIpTpCopyRxData
PDUR	PduR_SomeIpTpCopyTxData
PDUR	PduR_SomeIpTpRxIndication
PDUR	PduR_SomeIpTpStartOfReception
PDUR	PduR_SomeIpTpTransmit
PDUR	PduR_SomeIpTpTxConfirmation
ECUM	EcuM_BswErrorHook

Table 5-7 Services used by the SomeIpTp

5.3 Callback Functions

This chapter describes the callback functions that are implemented by the SomeIpTp and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `SomeIpTp_Cbk.h` by the SomeIpTp.

5.3.1 SomeIpTp_TriggerTransmit

Prototype	
`Std_ReturnType` SomeIpTp_TriggerTransmit `( PduIdType TxPduId, PduInfoType* *PduInfoPtr )`	
Parameter	
`TxPduId [in]`	ID of the PDU requested to be transmitted.

`PduInfoPtr [in][out]`	Contains a pointer to the buffer where the SDU data shall be copied. On return, the service will indicate the length of the data copied.
Return code	
`Std_ReturnType`	E_OK
	E_NOT_OK : SDU data has not been copied.
Functional Description	
Within this API, the called upper layer module shall check whether the available data fits into the buffer size reported by PduInfoPtr->SduLength. If it fits, it shall copy its data into the buffer provided by PduInfoPtr->SduDataPtr and update the length of the actual copied data. If not, it returns E_NOT_OK without changing PduInfoPtr.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant.	
Expected Caller Context	
> TASK|ISR2	

Table 5-8 SomeIpTp_TriggerTransmit

5.3.2 SomeIpTp_TxConfirmation

Prototype	
`void` SomeIpTp_TxConfirmation `( PduIdType TxPduId )`	
Parameter	
`TxPduId [in]`	ID of the PDU that has been transmitted.
Return code	
`void`	none
Functional Description	
The lower layer communication interface module confirms the transmission of a PDU.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant.	
Expected Caller Context	
> TASK|ISR2	

Table 5-9 SomeIpTp_TxConfirmation

5.3.3 SomeIpTp_RxIndication

Prototype	
`void` SomeIpTp_RxIndication `( PduIdType RxPduId, const PduInfoType* PduInfoPtr )`	
Parameter	
`RxPduId [in]`	ID of the received PDU.
`PduInfoPtr [in]`	Contains the length(SduLength) of the received PDU, a pointer to a buffer (SduDataPtr) containing the PDU.
Return code	
`void`	none
Functional Description	
Indication of a received PDU from a lower layer communication interface module.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is reentrant.	
Expected Caller Context	
> TASK|ISR2	

Table 5-10 SomeIpTp_RxIndication

5.4 Configurable Interfaces

None.

6 Configuration

The SomeIpTp attributes can be configured with the following tool:

> Configuration in DaVinci Configurator Pro 5

6.1 Configuration Variants

The SomeIpTp supports the configuration variants

> `VARIANT-PRE-COMPILE`

The configuration classes of the SomeIpTp parameters depend on the supported configuration variants. For their definitions please see the SomeIpTp_bswmd.arxml file.

6.2 Additional configuration hints

6.2.1 SomeIpTpRxTimeoutTime

The SomeIpTp module starts the timer after the first segment is received and waits for all the segmented PDUs of an SDU until this period before aborting the assembly process as specified in [1].

To avoid unpredictable interruption, this configured time should be sufficient to receive all segmented PDUs of an SDU.

6.2.2 SomeIpTpTxConfirmationTimeoutTime

The SomeIpTp module starts the timer immediately after a PDU is requested for transmission to the lower layer communication module and waits for transmission confirmation until this period before aborting the disassembly process.

To avoid unpredictable interruption, this configured time should be sufficient to transmit and process the confirmation of a segmented PDU. It should be less than `SomeIpTpSeparationTime`.

6.2.3 Sending and transmitting data without the RTE API

This guide is based on the provided templates listed in Table 4-3, and assumes that the routing paths between SoAd/SomeIpTp and LdCom/SomeIpTp are properly configured.

Make sure none of the LdCom's I-PDUs related to the SomeIpTp's SDUs have been mapped to the RTE, since this could trigger validation errors in the DaVinci Configurator and generation won't be possible.

6.2.3.1 Configuring the LdCom with the DaVinci Configurator

To receive data using the provided fictive component, the following parameters of the LdCom's Rx I-PDU related to the SomeIpTp's Rx SDU should be modified:

- LdComApiType: Set to `LDCOM_TP`.
- LdComRxCopyRxData: Set to the proper callback function. If the provided code is used, this would be `Cdd_SomeIpTp_LdComCbkCopyRxData`.

- LdComRxStartOfReception: Set to the proper callback function. If the provided code is used, this would be `Cdd_SomeIpTp_LdComCbkRxStartOfReception`.
- LdComTpRxIndication: Set to the proper callback function. If the provided code is used, this would be `Cdd_SomeIpTp_LdComCbkTpRxIndication`.

Figure 6-1 shows how the configuration should look like.

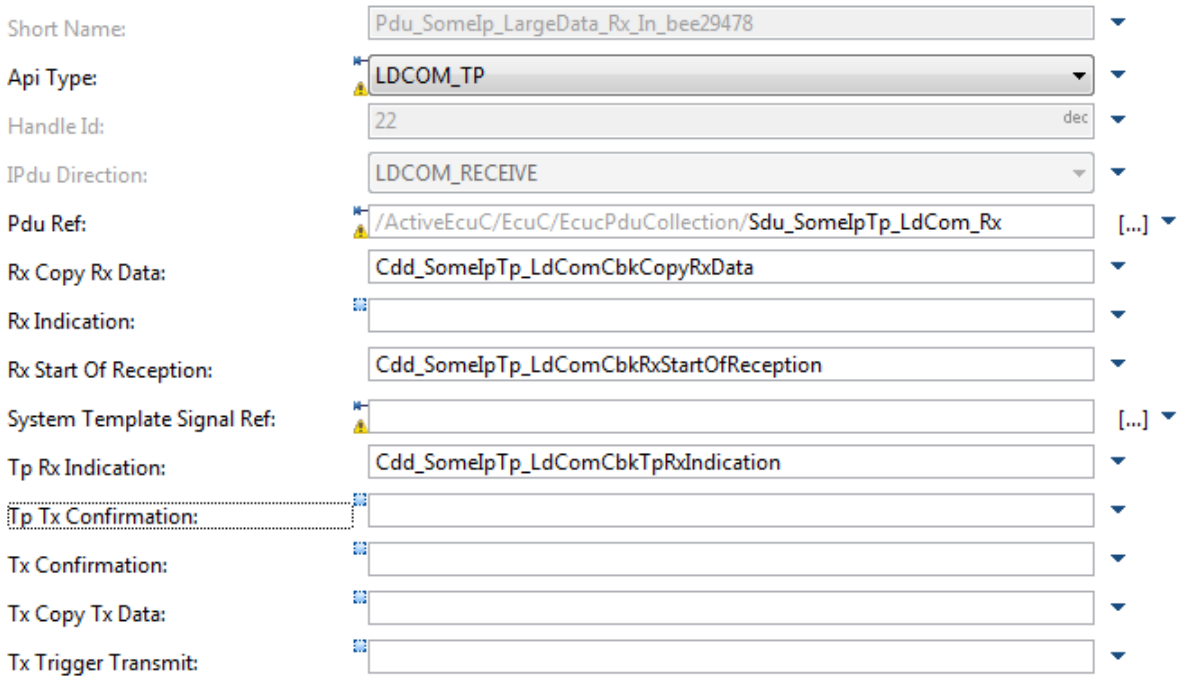


Figure 6-1 Configuration of the LdCom's I-PDU used during reception.

To transmit data using the provided fictive component, the following parameters of the LdCom's Tx I-PDU related to the SomeIpTp's Tx SDU should be modified:

- LdComApiType: Set to `LDCOM_TP`.
- LdComTpTxConfirmation: Set to the proper callback function. If the provided code is used, this would be `Cdd_SomeIpTp_LdComCbkTpTxConfirmation`.
- LdComTxCopyTxData: Set to the proper callback function. If the provided code is used, this would be `Cdd_SomeIpTp_LdComCbkCopyTxData`.

Figure 6-2 shows how the configuration should look like.

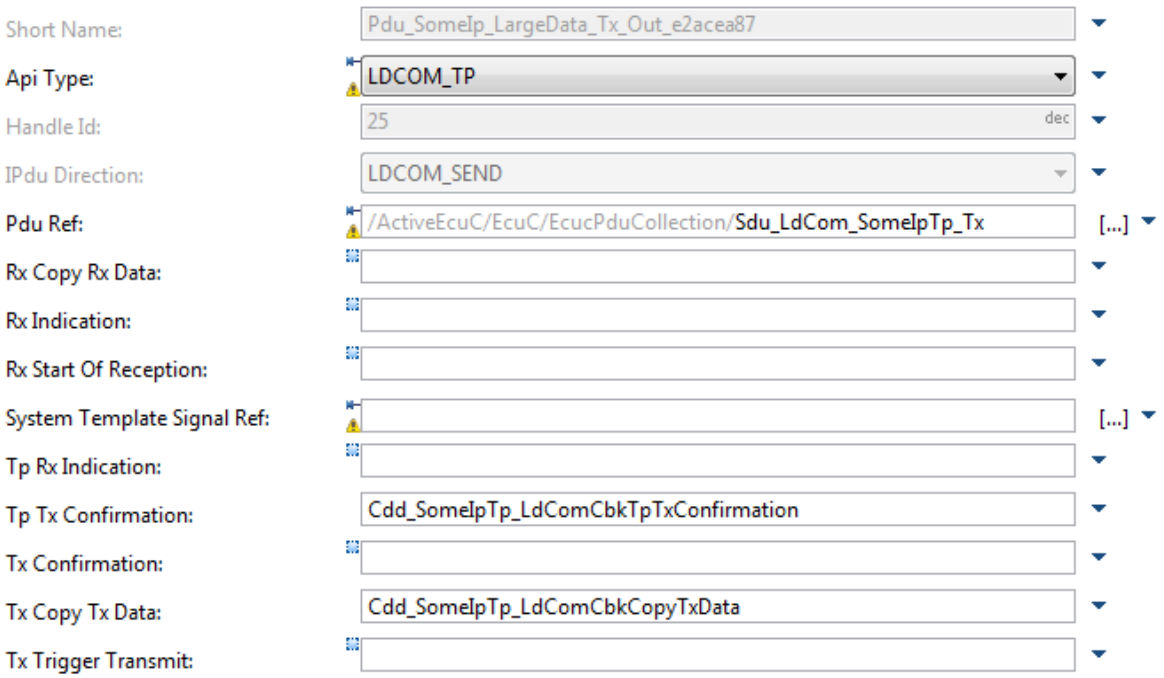


Figure 6-2 Configuration of the LdCom's I-PDU used during transmission.

Additionally, the header containing the declaration of the configured callback functions must be configured as an LdCom user config file (LdCom/LdComGeneral/LdComVersionInfoApi).

6.2.3.2 Adjusting the templates

The provided fictive component allows to transmit a stream of bytes over an LdCom Tx I-PDU. For this simple use case, the only mandatory change is to define the LdCom Tx I-PDU that shall be used for transmission in Cdd_SomeIpTp.h as follows:

```
#define CDD_SOMEIPTP_LDCOM_IPDU_TX LdComConf_LdComIPdu_Pdu_Symbolic_Name
```

LdCom_Cfg.h contains the symbolic names of the available LdCom I-PDUs.

Cdd_SomeIpTp.h also provides the array `Cdd_SomeIpTp_xfHeader`, which contains the SOME/IP header fields Request ID, Protocol Version, Interface Version, Message Type and Return Code of message to be sent. This fields can also be modified if required.

The comments in the templates provide more information.

6.2.3.3 Reading and Writing data from the SWC

Table 6-1 provides an overview of the functions provided by the fictive component that are intended to be used by the SWC. The functions are supposed to emulate the functionality provided by the RTE.

Function Name	Description
`Cdd_SomeIpTp_Read`	Reads the data received over the Rx I-PDU configured in 6.2.3.1.
`Cdd_SomeIpTp_IsUpdated`	Checks if the received data has been updated since the last read.
`Cdd_SomeIpTp_Write`	Writes data to be sent over the Tx I-PDU configured in 6.2.3.1.

Table 6-1 Functions intended to be used by the SWC.

The following example shows how the provided functions could be used within a runnable entity to receive, modify and send back a byte stream over the SomeIpTp.



Example

```
/**********************************************************************
 * DON'T CHANGE COMMENT!  <<Start of runnable>>  DON'T CHANGE COMMENT!
**********************************************************************/

#include "Cdd_SomeIpTp.h"

if (Cdd_SomeIpTp_IsUpdated() == TRUE)
{
  uint8 dst[CDD_SOMEIPTP_MAX_SDU_LENGTH];
  uint16 length = 0;

  if (Cdd_SomeIpTp_Read(dst, &length) == RTE_E_OK)
  {
    uint16 i;

    for (i = 0; i < length; i++)
    {
      dst[i]++;
    }
    Cdd_SomeIpTp_Write(dst, length);
  }
}
/**********************************************************************
 * DON'T CHANGE COMMENT!   <<End of runnable>>   DON'T CHANGE COMMENT!
**********************************************************************/
```

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
DaVinci Configurator Pro 5	Configuration and code generation tool for MICROSAR components

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SRS	Software Requirement Specification
SWS	Software Specification
PduR	PDU Router
PDU	Protocol Data Unit
SDU	Service Data Unit
EcuM	ECU State Manager
ECU	Electronic Control Unit

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2018 Vector Informatik GmbH